NVIDIA Kernels and Runtime Measurements

Deep Dive into DeepSeek — Chapter 35

Yin Yang

Foundation Books Project

The one rule of this chapter

Every number on these slides stays attached to the executed object that produced it.

- A GPU timing, a correctness gate, and a trace mean nothing on their own — they mean something only beside the hardware, the source revision, and the completion rule that made them safe to record.
- This chapter runs six dossiers — one CUDA calibration, DeepGEMM, FlashMLA, TileKernels, distributed DeepEP/EPLB/DualPipe, and matched serving — on real RTX 5090, H200, and Jetson Thor hardware.
- Nothing here is pooled into one benchmark number. Unlike objects keep unlike scopes.

Goal: follow one execution-evidence packet from a transparent CUDA tile through a complete four-H200 serving request, and see exactly which claim each recorded number can and cannot support.

Roadmap

1. The execution-evidence packet

Roadmap

1. The execution-evidence packet
2. A transparent CUDA baseline

Roadmap

1. The execution-evidence packet
2. A transparent CUDA baseline
3. DeepGEMM: from a static kernel to a generated one

Roadmap

1. The execution-evidence packet
2. A transparent CUDA baseline
3. DeepGEMM: from a static kernel to a generated one
4. FlashMLA: cache-aware attention

Roadmap

1. The execution-evidence packet
2. A transparent CUDA baseline
3. DeepGEMM: from a static kernel to a generated one
4. FlashMLA: cache-aware attention
5. TileKernels: a portability layer, honestly scoped

Roadmap

1. The execution-evidence packet
2. A transparent CUDA baseline
3. DeepGEMM: from a static kernel to a generated one
4. FlashMLA: cache-aware attention
5. TileKernels: a portability layer, honestly scoped
6. Distributed execution: DeepEP, EPLB, DualPipe

Roadmap

1. The execution-evidence packet
2. A transparent CUDA baseline
3. DeepGEMM: from a static kernel to a generated one
4. FlashMLA: cache-aware attention
5. TileKernels: a portability layer, honestly scoped
6. Distributed execution: DeepEP, EPLB, DualPipe
7. Serving: TensorRT-LLM and SGLang on four H200s

Roadmap

1. The execution-evidence packet
2. A transparent CUDA baseline
3. DeepGEMM: from a static kernel to a generated one
4. FlashMLA: cache-aware attention
5. TileKernels: a portability layer, honestly scoped
6. Distributed execution: DeepEP, EPLB, DualPipe
7. Serving: TensorRT-LLM and SGLang on four H200s
8. What this chapter does and does not establish

The execution-evidence packet

Five fields, one packet, growing scope

Definition (Execution-evidence packet)
$P = (\mathcal{S}, \mathcal{A}, \mathcal{E}, \mathcal{M}, \mathcal{C})$: semantic operation, artifact identity, execution context, measurement record, and admitted conclusion.

- $\mathcal{S}$ names the operation, inputs, output, and invariant.

Five fields, one packet, growing scope

Definition (Execution-evidence packet)
$P = (\mathcal{S}, \mathcal{A}, \mathcal{E}, \mathcal{M}, \mathcal{C})$: semantic operation, artifact identity, execution context, measurement record, and admitted conclusion.

- $\mathcal{S}$ names the operation, inputs, output, and invariant.
- $\mathcal{A}$ fixes source revisions, symbols, binaries, hashes.

Five fields, one packet, growing scope

Definition (Execution-evidence packet)
$P = (\mathcal{S}, \mathcal{A}, \mathcal{E}, \mathcal{M}, \mathcal{C})$: semantic operation, artifact identity, execution context, measurement record, and admitted conclusion.

- $\mathcal{S}$ names the operation, inputs, output, and invariant.
- $\mathcal{A}$ fixes source revisions, symbols, binaries, hashes.
- $\mathcal{E}$ names hardware, runtime, topology, and the completion event that makes the output safe to consume.

Five fields, one packet, growing scope

Definition (Execution-evidence packet)
$P = (\mathcal{S}, \mathcal{A}, \mathcal{E}, \mathcal{M}, \mathcal{C})$: semantic operation, artifact identity, execution context, measurement record, and admitted conclusion.

- $\mathcal{S}$ names the operation, inputs, output, and invariant.
- $\mathcal{A}$ fixes source revisions, symbols, binaries, hashes.
- $\mathcal{E}$ names hardware, runtime, topology, and the completion event that makes the output safe to consume.
- $\mathcal{M}$ keeps the correctness gate, timing protocol, observations, and blockers together.

Five fields, one packet, growing scope

Definition (Execution-evidence packet)
$P = (\mathcal{S}, \mathcal{A}, \mathcal{E}, \mathcal{M}, \mathcal{C})$: semantic operation, artifact identity, execution context, measurement record, and admitted conclusion.

- $\mathcal{S}$ names the operation, inputs, output, and invariant.
- $\mathcal{A}$ fixes source revisions, symbols, binaries, hashes.
- $\mathcal{E}$ names hardware, runtime, topology, and the completion event that makes the output safe to consume.
- $\mathcal{M}$ keeps the correctness gate, timing protocol, observations, and blockers together.
- $\mathcal{C}$ states the admitted result, denominator, exclusions, and scope.

Five fields, one packet, growing scope

Definition (Execution-evidence packet)
$P = (\mathcal{S}, \mathcal{A}, \mathcal{E}, \mathcal{M}, \mathcal{C})$: semantic operation, artifact identity, execution context, measurement record, and admitted conclusion.

- $\mathcal{S}$ names the operation, inputs, output, and invariant.
- $\mathcal{A}$ fixes source revisions, symbols, binaries, hashes.
- $\mathcal{E}$ names hardware, runtime, topology, and the completion event that makes the output safe to consume.
- $\mathcal{M}$ keeps the correctness gate, timing protocol, observations, and blockers together.
- $\mathcal{C}$ states the admitted result, denominator, exclusions, and scope.

A blocked measurement stays in $\mathcal{M}$ and narrows $\mathcal{C}$ — it never gets silently dropped.

Measured versus diagnostic, held apart

Invariant (Observation attachment)
Every reported value stays attached to the executed object, its owner, its completion condition, its correctness gate, and its measured interval.

Label	Meaning
measured	a correctness-gated timing or trace exists
diagnostic	a build, selection, guard, or harness condition resolved, performance unmeasured

The measured hardware set is SM90 H200, SM120 RTX 5090, and SM110 Jetson Thor. No SM100 device is available anywhere in this chapter.

A transparent CUDA baseline

Two kernels, one operation, zero counters missing

$$C_{i,j} = \sum_{k=0}^{K-1} A_{i,k} B_{k,j}, \qquad 16 \times 16 \text{ thread blocks}$$

- Direct kernel: one thread, one output element, one global load per operand per step.
- Tiled kernel: two 16×16 shared-memory arrays, two block-wide barriers per reduction tile.

Surface	Direct	Tiled
CUDA-event time (μs)	5.043	3.437

Two kernels, one operation, zero counters missing

$$C_{i,j} = \sum_{k=0}^{K-1} A_{i,k} B_{k,j}, \qquad 16 \times 16 \text{ thread blocks}$$

- Direct kernel: one thread, one output element, one global load per operand per step.
- Tiled kernel: two 16×16 shared-memory arrays, two block-wide barriers per reduction tile.

Surface	Direct	Tiled
CUDA-event time (μs)	5.043	3.437
Achieved occupancy (%)	15.08	16.64

Two kernels, one operation, zero counters missing

$$C_{i,j} = \sum_{k=0}^{K-1} A_{i,k} B_{k,j}, \qquad 16 \times 16 \text{ thread blocks}$$

- Direct kernel: one thread, one output element, one global load per operand per step.
- Tiled kernel: two 16×16 shared-memory arrays, two block-wide barriers per reduction tile.

Surface	Direct	Tiled
CUDA-event time (μs)	5.043	3.437
Achieved occupancy (%)	15.08	16.64

Shape $(129, 131, 127)$, zero max abs/relative error, 81 blocks on 170 SMs. The tiled path is $1.47\times$ faster here — a tiny-grid diagnostic, not a device-wide roofline claim. [OBSERVED: RTX 5090 teaching-kernel counter follow-up]

DeepGEMM: from a static kernel to a generated one

First call versus warm launch

DeepGEMM turns M, N, K, layouts, and dtypes into a generated, cached, specialized kernel: select $\rightarrow$ generate CUDA $\rightarrow$ compile/cache cubin $\rightarrow$ load $\rightarrow$ launch.

H200, SM90 FP8, M=4096, N=2112, K=7168	Value
First synchronized call	16878.6 ms

First call versus warm launch

DeepGEMM turns M, N, K, layouts, and dtypes into a generated, cached, specialized kernel: select $\rightarrow$ generate CUDA $\rightarrow$ compile/cache cubin $\rightarrow$ load $\rightarrow$ launch.

H200, SM90 FP8, M=4096, N=2112, K=7168	Value
First synchronized call	16878.6 ms
Five warm repeats, mean	0.12456 ms

First call versus warm launch

DeepGEMM turns M, N, K, layouts, and dtypes into a generated, cached, specialized kernel: select $\rightarrow$ generate CUDA $\rightarrow$ compile/cache cubin $\rightarrow$ load $\rightarrow$ launch.

H200, SM90 FP8, M=4096, N=2112, K=7168	Value
First synchronized call	16878.6 ms
Five warm repeats, mean	0.12456 ms
Ratio	$\approx 148{,}000\times$

First call versus warm launch

DeepGEMM turns M, N, K, layouts, and dtypes into a generated, cached, specialized kernel: select $\rightarrow$ generate CUDA $\rightarrow$ compile/cache cubin $\rightarrow$ load $\rightarrow$ launch.

H200, SM90 FP8, M=4096, N=2112, K=7168	Value
First synchronized call	16878.6 ms
Five warm repeats, mean	0.12456 ms
Ratio	$\approx$ 148,000$\times$

$\mathrm{calc\_diff} = 6.84 \times 10^{-4} < 10^{-3}$ passed before any timing was admitted. Nsight Compute returned `ERR_NVGPUCTRPERM`: no H200 hardware counter exists for this specialization. [OBSERVED: Panther H200 expanded precision and profiler packet]

From concept to code: the JIT cache is a rename, not a magic step

from DeepGEMM/csrc/jit/compiler.hpp

```
if (const auto runtime = kernel_runtime_cache->get(dir_path); runtime)
    return runtime;                          // warm: reuse, no generation
compile(code, tmp_dir_path, tmp_cubin_path);   // cold: generate + compile
std::filesystem::rename(tmp_dir_path, dir_path, error_code);
```

From concept to code: the JIT cache is a rename, not a magic step

from DeepGEMM/csrc/jit/compiler.hpp

```
if (const auto runtime = kernel_runtime_cache->get(dir_path); runtime)
    return runtime;                          // warm: reuse, no generation
compile(code, tmp_dir_path, tmp_cubin_path);   // cold: generate + compile
std::filesystem::rename(tmp_dir_path, dir_path, error_code);
```

The cache hit at the top is the whole reason a warm call skips 148,000× of work. The atomic rename is the safe stage-release event that publishes a finished cubin to every future caller.

[INSPECTED: DeepGEMM csrc/jit/compiler.hpp]

FlashMLA: cache-aware attention

Same output, three concerns: score, cache, timing

$$s_i = \alpha q k_i^{\mathsf{T}}, \qquad p_i = \frac{e^{s_i}}{\sum_j e^{s_j}}, \qquad o = \sum_i p_i v_i$$

- The SM90 dense-decode kernel splits a 64×576 cache tile into nine 64×64 TMA-fed slices with online-softmax state passed between warpgroups.
- H200 dense decode: 0.0419 ms mean (fixed cache); sparse decode: 0.0993–0.0938 ms mean; all rows passed both output *and* LSE gates.

The live integration probe found 348 of 864 same-input comparisons failing an elementwise tolerance while LSE passed every time — correctness is checked object by object, not folded into one pass/fail count. [OBSERVED: Panther H200 integrated FlashMLA and DeepGEMM route]

TileKernels: a portability layer, honestly scoped

Import, correctness, and timed launch are three different claims

Device	State	What is admitted
H200, SM90	measured	per-token cast and fused-MoE-mapping timings

Import, correctness, and timed launch are three different claims

Device	State	What is admitted
H200, SM90	measured	per-token cast and fused-MoE-mapping timings
RTX 5090, SM120	correctness-only	4/4 and 4/4 rows pass; no timing

Import, correctness, and timed launch are three different claims

Device	State	What is admitted
H200, SM90	measured	per-token cast and fused-MoE-mapping timings
RTX 5090, SM120	correctness-only	4/4 and 4/4 rows pass; no timing
Thor, SM110	diagnostic	3-sample timings; one non-positive SM-count harness closure

Import, correctness, and timed launch are three different claims

Device	State	What is admitted
H200, SM90	measured	per-token cast and fused-MoE-mapping timings
RTX 5090, SM120	correctness-only	4/4 and 4/4 rows pass; no timing
Thor, SM110	diagnostic	3-sample timings; one non-positive SM-count harness closure

A successful import is weaker than a correctness-gated launch, which is weaker than a timed production call. TileLang compiles to a target and a device — portability is a claim about each pairing, not the source once. [OBSERVED: TileKernels device dossier, H200/RTX 5090/Thor]

Distributed execution: DeepEP, EPLB, DualPipe

Concurrency observed; speedup not claimed

- DeepEP dispatch packs routed tokens, publishes them with a release–acquire tail handoff, and combine restores origin-token order.
- Two-node, four-H200 model-shaped fixture: independent GPU work intersected dispatch by 274–309 μs in all eight asynchronous ranges; dependent MLP work still waited for dispatch, and combine still waited for the MLP result.
- DualPipe's complete-schedule run matched the full oracle exactly, yet its concurrent medians were 1.95–3.80% *slower* than serialized.

Colored timeline overlap proves temporal intersection. It does not, by itself, prove that the critical path got shorter. [OBSERVED: Panther H200 DeepEP Nsight Systems scale-out]

Serving: TensorRT-LLM and SGLang on four H200s

Matched request, matched hardware, opposite tails

Fixed: DeepSeek-R1-Distill-Qwen-32B, BF16, TP4, four H200s, same tokenized requests.
Changed: the runtime.

Concurrency	tokens/s		p99 TTFT (ms)	
	TRT-LLM	SGLang	TRT-LLM	SGLang
1	14.4	136.5	11,162.1	166.9

Matched request, matched hardware, opposite tails

Fixed: DeepSeek-R1-Distill-Qwen-32B, BF16, TP4, four H200s, same tokenized requests.
Changed: the runtime.

Concurrency	tokens/s		p99 TTFT (ms)	
	TRT-LLM	SGLang	TRT-LLM	SGLang
1	14.4	136.5	11,162.1	166.9
32	185.8	1,162.2	25,537.0	3,016.2

Matched request, matched hardware, opposite tails

Fixed: DeepSeek-R1-Distill-Qwen-32B, BF16, TP4, four H200s, same tokenized requests.
Changed: the runtime.

Concurrency	tokens/s		p99 TTFT (ms)	
	TRT-LLM	SGLang	TRT-LLM	SGLang
1	14.4	136.5	11,162.1	166.9
32	185.8	1,162.2	25,537.0	3,016.2

SGLang's throughput and TTFT both look better at every tested concurrency, yet at concurrency 32 its p99 TPOT is 1.61× TensorRT-LLM's. Queueing/prefill and decode cadence can move in opposite directions — the TensorRT-LLM route used the v1.2.1 PyTorch backend, not a built engine. [OBSERVED: Panther H200 matched TensorRT-LLM and SGLang serving comparison]

What this chapter does and does not establish

The open list is part of the result

Question	State
H200 DeepGEMM hardware counters	blocked by `ERR_NVGPUCTRPERM`
Matched SM90/SM100 execution	unmeasured; no SM100 device available
FlashMLA attention on SM120/SM110	guards reached; no supported metric row
TensorRT-LLM path attribution	serialized-engine and stage-resolved timing unmeasured

A blocked or unmeasured row defines scope. It is never quietly replaced by a neighboring device's number or a source-only expectation. [OPEN: closed unavailable work, chapter source basis]

Concept-to-code map for the chapter

- **execution-evidence packet fields** → `code/chapter_expert_nvidia_developer_track/evidence/lab-protocols.json`
- **CUDA calibration kernels** → `code/chapter_expert_nvidia_developer_track/src/profiled_gemm.cu`
- **JIT compile/cache/release** → `DeepGEMM/csrc/jit/compiler.hpp`
- **output-and-LSE correctness gate** → `FlashMLA/tests/test_flash_mla_dense_decoding.py`
- **exact dispatch/combine assertions** → `DeepEP/tests/elastic/test_ep.py`

Keep every number attached to what ran, on what hardware, checked how.

Next: ten sections walk these six dossiers in depth, then the cross-system synthesis and exercises close the chapter.